

SentinelAI

**Agentic Threat Intelligence and Human-Approved Response for Suspicious URLs,
IPs and Files**

**A MINI PROJECT REPORT FOR THE COURSE
AD23731 INTRODUCTION TO AGENTIC AI**

Submitted by

**Aniruth S V - 231501015
Girivasanth V - 231501048**

IV YEAR B.Tech



**Department of Artificial Intelligence and Machine Learning
RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai – 602 105**

OCTOBER 2026



BONAFIDE CERTIFICATE

Certified that the mini project titled "**SentinelAI - Agentic Threat Intelligence and Human-Approved Response for Suspicious URLs, IPs and Files**" is the bona fide work carried out by **Aniruth S V(231501015) and Girivasanth V(231501048)** of Year -IV, VII Semester, **B.Tech. Artificial Intelligence and Machine Learning Department**. This project has been completed in partial fulfilment of the requirements of the course **AD23731 - Introduction to Agentic AI**, during the Academic Year 2026-2027 promoting project-based learning and practical application of Agentic AI.

Faculty In-Charge

Submitted to mini-project viva voce examination held on:

Internal Examiner

External Examiner

Table of Contents

Content:

Section	Topic	Page
1.	Introduction	2
	1.1 Background and Motivation	2
	1.2 Need for Agentic AI	2
	1.3 Project Objectives	3
2.	Problem Statement and Business Case	3
	2.1 Problem Definition	3
	2.2 Business / Real-World Context	4
3.	User and Stakeholder Analysis	4
	3.1 Target Users	4
	3.2 Stakeholder Identification	4
	3.3 User Requirements	5
4.	Proposed Agentic AI Solution	5
	4.1 Overall Solution	5
	4.2 Key Features	6
	4.3 Agent-Based Approach	6
5.	Dataset / Knowledge Sources	6
	5.1 Dataset / Knowledge Source Description	6
	5.2 Evaluation Dataset	7
	5.3 Tools and Technologies Used	7
6.	Agent Architecture	8
	6.1 Agent Components	9
	6.2 Tools	10
	6.3 Memory	10
	6.4 Reasoning Flow	10
	6.5 Agent Interactions	10
7.	Agent Workflow and Orchestration	11
	7.1 Workflow Design	11
	7.2 Task Decomposition	11
	7.3 Agent Coordination	11
	7.4 Orchestration Implementation	12
8.	Agentic RAG Implementation	12
	8.1 Retrieval Architecture	12
	8.2 Knowledge Retrieval	12
	8.3 Generation Process	13
	8.4 Retrieval-Quality Evaluation	13
9.	System Implementation	13
	9.1 Functional Prototype Screenshots	13
	9.2 Source-Code Repository	15
10.	Experimental Evaluation and Results	15
	10.1 Evaluation Metrics	15
	10.2 Testing Methodology	16
	10.3 Performance Results	16
	10.4 Success-Metric Achievement	18

11.	Comparison, Innovation and Scalability	19
	11.1 Comparison with Baseline	19
	11.2 Advantages of the Proposed System	20
	11.3 Innovation	20
	11.4 Scalability	20
	11.5 Real-World Applicability	20
12.	Conclusion and Future Work	21

Tables:

Table	Caption	Page
Table 1	Stakeholder map and interests.	4
Table 2	Curated cybersecurity knowledge base entries used by the RAG agent.	7
Table 2A	Phish-IRIS validation subset used for screenshot-classification evaluation.	7
Table 3	Technology stack by architectural layer.	8
Table 4	Published risk thresholds implemented in decision_from_scores; any BLOCK action, or confidence below 0.65, forces analyst approval regardless of score.	12
Table 5	Live RAG retrieval output for a real MEDIUM-risk investigation, demonstrating that the highest-overlap, most topically relevant reference (CISA phishing indicators) is correctly ranked first.	13
Table 6	Repository layout.	15
Table 7A	Classification metrics for the selected Phish-IRIS validation subset.	17
Table 7	Live evaluation results across four representative indicators (benign infrastructure, a benign well-known site, and two synthetically suspicious URLs).	18
Table 8	Mapping of stated objectives to demonstrated evidence.	19
Table 9	Baseline comparison across the dimensions that matter for SOC adoption.	19

Figures:

Figure	Caption	Page
Figure 1	SentinelAI system architecture: client surfaces, API layer, agent orchestration, external evidence providers, human-in-the-loop response, and the local-to-Azure deployment path.	8
Figure 2	The LangGraph investigation graph: a linear StateGraph over a shared InvestigationState, terminating at the orchestrator before human approval.	11
Figure 3	Investigation Console populated with four real investigations produced by a local run of the backend: two benign (LOW), one MEDIUM awaiting approval, and one HIGH that was subsequently approved and moved to response_simulated.	14
Figure 4	Approval Queue detail view for the HIGH-risk case, reproducing the exact explanation, provider-status and RAG-reference JSON returned by POST /api/v1/investigations for this indicator.	15

SentinelAI

Agentic Threat Intelligence and Human-Approved Response for Suspicious URLs, IPs and Files

Final Project Report
Agentic AI Systems

Project repository: [agentic-threat-intelligence](#) (SentinelAI MVP)
Report prepared: September 2026

1. Introduction

1.1 Background and Motivation

Phishing and malicious-infrastructure attacks remain one of the most common initial-access techniques used against organisations. Adversaries stand up look-alike login pages, credential-harvesting forms and malware-hosting URLs faster than manual security-operations-centre (SOC) processes can verify them. A single Tier-1 analyst may need to open a suspicious link in an isolated browser, cross-reference three or four reputation feeds, inspect the rendered page for brand impersonation, and finally decide whether to escalate a block — a workflow that can take several minutes per indicator and does not scale to the volume of links reported by users or automatically extracted from email and browsing telemetry.

SentinelAI, the system analysed in this report, is a working prototype that automates this investigation while keeping a human analyst in control of any containment action. It was built to demonstrate how an agentic AI architecture — a graph of cooperating, tool-using agents rather than a single prompt-response model — can decompose a multi-step security investigation into specialised stages: input validation, threat-intelligence lookup, visual (screenshot) analysis, knowledge retrieval, and risk scoring, before pausing for a documented, auditable human decision.

1.2 Need for Agentic AI

A conventional single-call classifier ("is this URL malicious, yes or no") cannot satisfy the requirements of a SOC workflow for three reasons that motivate the agentic design used here:

- Multi-source evidence gathering. A defensible verdict needs several independent signals — URL/domain structure, third-party reputation databases (VirusTotal, AbuseIPDB, URLScan), and the visual content of the page itself. Each of these is a distinct tool call with its own failure mode, and an agent must decide which tools are even applicable to a given indicator type (a file hash cannot be screenshotted; an IP address has no visual analysis).
- Sequential, stateful reasoning. The visual analysis prompt benefits from knowing what the threat-intelligence stage already found; the final score explanation must reference which knowledge-base entries and which provider results actually contributed. This requires a shared, evolving state object that is passed between reasoning steps, not a single stateless completion.
- Bounded autonomy with human oversight. Because a false positive can block legitimate traffic and a false negative can allow an active phishing campaign, the system is deliberately not permitted to act autonomously beyond scoring and recommending — it must stop and request analyst approval before any containment action is recorded, and every step must be explainable and auditable after the fact.

These three properties — tool orchestration, shared state across steps, and a human-in-the-loop (HITL) checkpoint — are the defining characteristics of an agentic AI system, and they are the reason the project is built on LangGraph's explicit state-graph orchestration rather than a single large-language-model prompt.

1.3 Project Objectives

- Design and implement a multi-agent investigation pipeline that ingests a public URL, IP address, or SHA-256 file hash and produces an explainable risk score.
- Integrate live, credential-gated threat-intelligence providers (VirusTotal, AbuseIPDB, URLScan) alongside deterministic local heuristics so the system degrades gracefully when external keys are not configured, instead of fabricating evidence.
- Add a vision-language-model (VLM) agent that inspects an automatically captured, SSRF-guarded screenshot of a page for phishing indicators such as login forms and brand impersonation.
- Implement an agentic Retrieval-Augmented Generation (RAG) stage that grounds every verdict in cited cybersecurity references (MITRE ATT&CK, OWASP, CISA, NIST) drawn from a curated knowledge base, with an Azure AI Search-backed live path for production scale.
- Provide a transparent, weighted scoring formula with a full factor-by-factor explanation, confidence estimate, and documented limitations, rather than an opaque single number.
- Enforce a human-in-the-loop approval gate for any block-level recommendation, backed by a persistent, append-only audit trail and a removable response blocklist.
- Ship the workflow through three real surfaces — a React SOC dashboard, a Chrome/Edge Manifest V3 browser extension, and a documented REST API — and provide a path from local Docker Compose deployment to an Azure production topology.

2. Problem Statement and Business Case

2.1 Problem Definition

SOC teams are required to triage a continuous stream of potentially malicious indicators — links reported by employees, URLs seen in email gateways, IP addresses flagged by network telemetry, and file hashes from endpoint tooling. For each indicator, an analyst must answer four questions before any action can be taken: (1) What does third-party threat intelligence say about this indicator? (2) If it is a web page, does it visually resemble a phishing attempt (login form, brand impersonation, urgency cues)? (3) What documented attack technique or guidance does this resemble, so the decision can be justified? (4) What is the appropriate response, and who approved it? Doing this manually and consistently, indicator after indicator, at the volume modern organisations face, is not sustainable with a fixed analyst headcount, yet fully automated blocking is unacceptably risky without human oversight, because a wrong automatic block can take down legitimate business traffic.

2.2 Business / Real-World Context

The cost of this gap shows up in two directions. Under-investigation lets phishing and malware-hosting infrastructure remain reachable long enough to cause credential theft or a ransomware foothold — MITRE ATT&CK technique T1566 (Phishing) and T1583.001 (acquiring look-alike domains) are among the most common initial-access techniques used against organisations of every size. Over-investigation, on the other hand, consumes analyst time on low-risk indicators that structural heuristics could have triaged automatically. SentinelAI's business case is to shift analyst effort toward the decisions that actually require judgement — approving, rejecting, or monitoring a small number of medium/high-risk cases each day — while the agentic pipeline performs the repeatable evidence-gathering work, produces a defensible, explainable record for every investigation (useful for compliance and post-incident review under frameworks such as NIST SP 800-61r3 incident-response guidance), and gives ordinary end-users a lightweight browser extension that surfaces the same risk assessment before they interact with a page.

3. User and Stakeholder Analysis

3.1 Target Users

- Tier-1/Tier-2 SOC analysts who submit indicators for investigation and act on the approval queue.
- Security engineers / SOC leads who monitor system readiness, manage the response blocklist, and review the audit trail during incident post-mortems.
- General employees / end-users who install the browser extension and receive an automatic risk badge while browsing, without needing to understand the underlying agent pipeline.

3.2 Stakeholder Identification

Stakeholder	Interest in the system
SOC analysts (primary users)	Faster, evidence-backed triage; a single approval queue instead of manually querying multiple tools.
Security / IT leadership	Defensible, auditable decisions; reduced mean-time-to-triage; visibility into provider coverage via the System Status view.
Compliance / audit function	An immutable, timestamped audit trail of every investigation, decision and blocklist change for post-incident and regulatory review.
End-users / employees	A non-intrusive browser warning before interacting with a risky page, without exposing raw file contents (only a SHA-256 hash is transmitted for file scans).
Platform / DevOps engineers	A containerised, Azure-portable deployment (Docker Compose locally, Container Apps / managed Postgres / Blob Storage / AI Search in production) with clear configuration boundaries.

Table 1 — Stakeholder map and interests.

3.3 User Requirements

Functional requirements

- Accept a URL, IP address, or SHA-256 hash and return a verdict (LOW/MEDIUM/HIGH/CRITICAL) with a numeric score and recommended action.
- Automatically capture a screenshot of submitted URLs (opt-out available) and analyse it for phishing indicators.
- Surface supporting cybersecurity references for every verdict, with an explanation of why each was retrieved.
- Require analyst approval before any BLOCK recommendation is enacted, and record approve/reject/monitor decisions with a comment.
- Maintain a removable, persistent blocklist and a complete, queryable audit trail.
- Provide the same analysis to a browser extension via a token-authenticated endpoint, and support local file-hash scanning without uploading file contents.

Non-functional requirements

- Safe-by-default: never fabricate a verdict from a provider that is not configured; show it as "not_configured" instead.
- SSRF-resistant: reject private, loopback, link-local and reserved network destinations at multiple layers (indicator normalisation, DNS resolution, and screenshot navigation).
- Explainable: every score must be traceable to a weighted formula and named contributing factors, not an opaque model output.
- Portable: run unmodified from a local Docker Compose stack to an Azure Container Apps deployment.

4. Proposed Agentic AI Solution

4.1 Overall Solution

SentinelAI is a modular-monolith FastAPI service that exposes a REST API consumed by a React SOC dashboard and a Manifest V3 browser extension. At its core is a LangGraph state graph — five cooperating agents that validate an indicator, gather threat intelligence, analyse a screenshot with a vision-language model, retrieve supporting cybersecurity knowledge, and combine everything into a transparent, weighted risk score. High-risk or low-confidence results are queued for human approval; approved indicators are added to a persistent, removable blocklist, and every step is written to an append-only audit trail.

4.2 Key Features

- Multi-agent LangGraph investigation workflow (validation → threat intelligence → visual analysis → RAG → risk scoring).
- Live provider routing to VirusTotal, AbuseIPDB and URLScan when API keys are configured, with deterministic local URL-structure heuristics that work with zero configuration.
- Automatic, SSRF-guarded website screenshot capture and Azure vision-language-model analysis of the rendered page.
- Explainable scoring: factor weights, per-factor contributions, threshold rationale, confidence rationale, and explicit "limitations" text when evidence is missing.
- Curated local cybersecurity knowledge base with an optional Azure AI Search-backed live retrieval path, each result annotated with why it is relevant.
- Human approve / reject / monitor decision flow with analyst comments, a persistent removable blocklist, and full audit logging.
- A fully interactive React dashboard (Investigations, Approval Queue, Blocklist, Audit Trail, System Status) and a Chrome/Edge extension with automatic active-page analysis, a toolbar risk badge, and local SHA-256 file hashing.

4.3 Agent-Based Approach

Rather than a single model call, the workflow is decomposed into five narrowly scoped agents that each own one responsibility and hand off a shared, typed state object (`InvestigationState`). This mirrors how a human analyst actually works — validate the input, gather intelligence, look at the page, check documented guidance, then weigh the evidence — and it lets each stage fail independently and transparently: if VirusTotal is not configured or URLScan returns an HTTP error, that specific agent records the failure in the evidence rather than silently degrading the whole investigation. Section 6 details the architecture of each agent.

5. Dataset / Knowledge Sources

5.1 Dataset / Knowledge Source Description

SentinelAI does not fine-tune or retrain its vision-language model. For quantitative evaluation, the project uses only the Phish-IRIS validation dataset and draws a 550-screenshot subset containing 350 legitimate pages and 200 phishing pages. During operation, the system's knowledge is assembled separately from live threat-intelligence responses and a curated cybersecurity reference corpus used for retrieval-augmented grounding. The curated knowledge base contains five reference entries embedded in the RAG agent and can be replaced by Azure AI Search at production scale:

ID	Source	Used to ground
MITRE-T1566	MITRE ATT&CK T1566 — Phishing	Credential-harvesting links, cloned sign-in pages, malicious attachments.
MITRE-T1583.001	MITRE ATT&CK T1583.001 — Acquire Infrastructure: Domains	Typosquatted / brand-impersonating domain registrations.
OWASP-SSRF	OWASP SSRF Prevention Cheat Sheet	Destination-validation rationale for the screenshot and IP-resolution safeguards.
NIST-IR	NIST SP 800-61r3 Incident Response Guidance	Containment/response and evidence-preserving decision rationale.
CISA-PHISHING	CISA Phishing Recognition Guidance	Behavioural phishing indicators (urgency, mismatched domains, credential requests).

Table 2 — Curated cybersecurity knowledge base entries used by the RAG agent.

Each reference entry carries a title, authoritative summary, keyword list and source URL. Runtime evidence is assembled from the submitted indicator, resolved public IP addresses, VirusTotal, AbuseIPDB and URLScan responses, the captured screenshot, and the top-matching knowledge-base entries. The Phish-IRIS validation subset is used only to measure screenshot classification performance; it is not used to train model weights.

5.2 Evaluation Dataset

The evaluation subset was drawn exclusively from the Phish-IRIS validation split. The selected 550 screenshots preserve both target classes and were not used for model training.

Dataset	Split	Selection	Legitimate	Phishing	Total
Phish-IRIS	Validation	Selected subset	350	200	550

Table 2A - Phish-IRIS validation subset used for screenshot-classification evaluation.

5.3 Tools and Technologies Used

Layer	Technology
Agent orchestration	LangGraph 0.6 (StateGraph), Python 3.12
Backend API	FastAPI 0.116, Pydantic-Settings, Uvicorn
Persistence	SQLAlchemy 2.0, PostgreSQL 17 (SQLite for local/test runs)
Threat intelligence	VirusTotal API v3, AbuseIPDB API, URLScan.io API (httpx client)

Layer	Technology
Visual analysis (VLM)	Azure OpenAI vision-capable deployment through the AzureOpenAI chat-completions client; local Ministral 3 inference is not implemented in the current codebase
Screenshot capture	Playwright (Chromium), isolated browser context, navigation/redirect revalidation
Retrieval-augmented generation	Curated local keyword-matching index; optional Azure AI Search (azure-search-documents)
Object storage	Local evidence directory / Docker evidence volume; optional Azure Blob Storage or Azurite through azure-storage-blob
Frontend	React 18 + TypeScript, Vite, hand-written CSS design system
Browser extension	Chrome/Edge Manifest V3 service worker, Web Crypto API (local SHA-256 hashing)
Containerisation & IaC	Docker Compose (Postgres, Redis reserved for future queue/cache use, Azurite, backend, frontend); Azure Bicep for Storage, Container Registry and Log Analytics
Testing	Pytest 8.4, FastAPI TestClient, SQLAlchemy StaticPool in-memory SQLite

Table 3 — Technology stack by architectural layer.

6. Agent Architecture

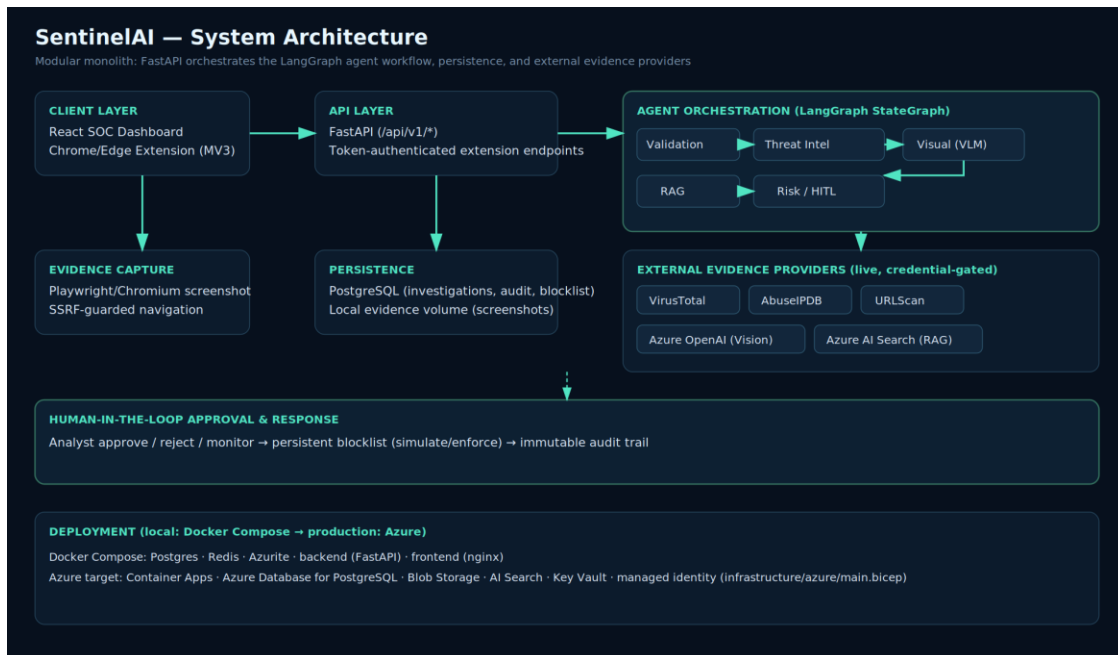


Figure 1 — SentinelAI system architecture: client surfaces, API layer, agent orchestration, external evidence providers, human-in-the-loop response, and the local-to-Azure deployment path.

6.1 Agent Components

6.2 Tools

Each agent calls a small set of narrowly scoped tools rather than a general-purpose function-calling loop, which keeps the pipeline deterministic and auditable:

- `resolve_public_ips` — DNS resolution with SSRF re-validation (`security.py`).
- `ThreatIntelClients.virustotal_url/ip/hash`, `.abuseipdb`, `.urlscan_lookup` — external reputation lookups (`integrations/threat_intel.py`), each wrapped in a `safely()` helper that converts any exception into a structured `{status: "error"}` result instead of crashing the investigation.
- `capture_url_screenshot` — Playwright-driven, SSRF-guarded screenshot capture with navigation/redirect revalidation, an isolated browser context, and configurable timeout/settle/size limits (`screenshot_capture.py`).
- `azure_openai_client(...).chat.completions.create` - AzureOpenAI call to the configured Azure vision-capable deployment.
- `azure_search_client(...).search` — optional Azure AI Search query, with a local fallback if the call raises.
- `decision_from_scores` — the deterministic scoring/explanation function used by the orchestrator (`live_analysis.py`).

6.3 Memory

The project uses three distinct memory scopes rather than a single conversational memory buffer, which is appropriate for an auditable security workflow:

- Working memory: the `InvestigationState` TypedDict passed node-to-node inside a single `LangGraph` run — `indicator`, `resolved_ips`, `threat_intel`, `visual_analysis`, `rag_context`, `risk_score`, and an `agent_trace` list that every node appends a step to.
- Episodic / audit memory: each investigation, screenshot event, analyst decision, and blocklist change is written as an immutable `AuditEvent` row in PostgreSQL, giving a durable record that outlives the process and supports post-incident review.
- Persistent case memory: the `Investigation`, `BlocklistEntry` and screenshot-evidence records themselves, which let the dashboard re-open any past case with its full evidence and let an analyst act on cases hours or days after they were opened.

6.4 Reasoning Flow

The reasoning flow is intentionally deterministic rather than free-form chain-of-thought: only the VLM node asks a language model to reason (about the pixels in a screenshot); every other decision — which providers to call, how to weight evidence, which verdict threshold applies, why a knowledge-base entry was retrieved — is computed by transparent, unit-tested Python logic. This is a deliberate design trade-off for a security-response system: it sacrifices some of the open-ended flexibility of a

fully LLM-driven agent in exchange for a score that can be reproduced, unit-tested, and explained to a human analyst and, if needed, to an auditor.

6.5 Agent Interactions

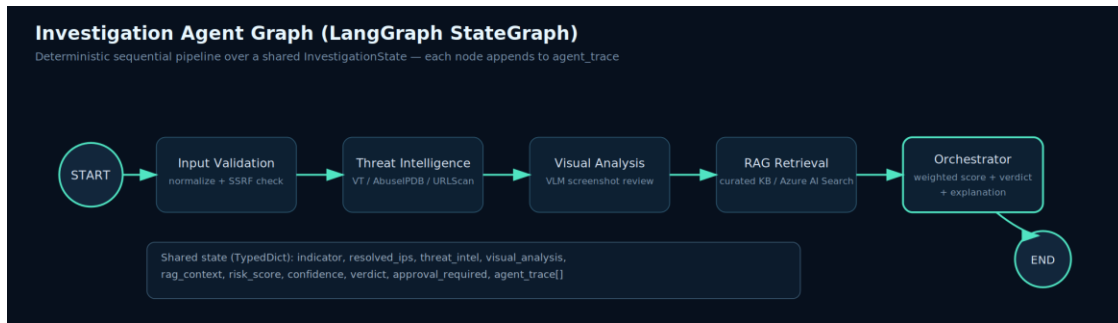


Figure 2 — The LangGraph investigation graph: a linear StateGraph over a shared InvestigationState, terminating at the orchestrator before human approval.

The graph is compiled once (`build_investigation_graph()`) and invoked synchronously per investigation via `investigation_graph.invoke(...)` from the service layer, which keeps latency predictable and makes the five-step trace easy to show verbatim in the dashboard's "Agent Trace" tab.

7. Agent Workflow and Orchestration

7.1 Workflow Design

The workflow is a single, linear StateGraph with five nodes and no conditional branching at the graph level (validate → threat_intelligence → visual_analysis → rag → risk → END). Branching instead happens inside each node based on `indicator_type` and configuration — for example, the VLM node substitutes a "not_applicable" result for IP/hash indicators rather than the graph routing around it. This keeps the graph topology simple and testable while still producing indicator-appropriate behaviour.

7.2 Task Decomposition

Investigation is decomposed along the natural boundaries of the evidence types involved: identity/safety validation, external reputation, visual/page-content evidence, documented-knowledge grounding, and final synthesis. This mirrors Section 6.1 and lets each stage be developed, tested, and reasoned about independently — the test suite (Section 10) exercises threat-intelligence scoring, screenshot handling, and the security validator each in isolation, in addition to full end-to-end API tests.

7.3 Agent Coordination

Coordination is achieved purely through the shared InvestigationState dictionary that each node receives and partially updates — there is no message-passing or blackboard system beyond this typed state object, and node ordering is fixed by the graph edges rather than negotiated at run time. Two

implementation details are notable: first, `threat_intel_node` and `vlm_node` perform local (function-scope) imports of `live_analysis` to avoid a circular import, since `live_analysis` itself reuses the URL keyword weights defined in `nodes.py`; second, every node appends a structured entry to `agent_trace` via the shared `append_trace()` helper, so the dashboard can render a literal, ordered log of what each agent did and found, in the agent's own words.

7.4 Orchestration Implementation

The orchestrator (`risk_node`) is the only node with cross-cutting responsibility: it does not gather new evidence, it synthesises what the other four nodes produced. Concretely, it calls `decision_from_scores(intel_score, visual_score, evidence_available, visual_available)` to obtain a score, verdict, action and base explanation, then rewrites the explanation's factor reasons using the actual provider names, statuses and matched indicators, appends explicit "limitations" text whenever a provider was unconfigured or errored, and finally sets `approval_required = True` whenever the recommended action is BLOCK or confidence is below 0.65 — the single line in the codebase that enforces the human-in-the-loop guarantee described in Sections 3 and 4.

Scoring formula (from `decision_from_scores`):

`score = round(intel_score × intel_weight + visual_score × visual_weight, 1)`

`intel_weight, visual_weight = (0.65, 0.35) if visual_available else (1.0, 0.0)`

Score range	Verdict	Recommended action
0 – 24.9	LOW	ALLOW
25 – 49.9	MEDIUM	MONITOR
50 – 69.9	HIGH	BLOCK (approval required)
70 – 100	CRITICAL	BLOCK (approval required)

Table 4 — Published risk thresholds implemented in `decision_from_scores`; any BLOCK action, or confidence below 0.65, forces analyst approval regardless of score.

8. Agentic RAG Implementation

8.1 Retrieval Architecture

The RAG agent implements a dual-path retrieval architecture. When Azure AI Search credentials are configured, it issues a live search against the configured index and normalises each hit into the same `{id, title, text, url, relevance_score, why_relevant}` shape used everywhere else in the system, wrapped in a `try/except` so an Azure outage falls back to the local path instead of failing the investigation. Otherwise — and by default in this MVP — it searches the five-entry curated knowledge base described in Section 5.1 directly inside the process.

8.2 Knowledge Retrieval

The local retrieval algorithm builds an evidence string from the indicator, page title, matched threat-intel keywords, and VLM-detected suspicious elements/brands; tokenises it with a simple `[a-z0-9]+` regex; and scores each knowledge-base entry by the size of the token overlap between the evidence and the entry's title/text/keywords, plus a +2 bonus for every configured keyword that appears verbatim in the evidence string. The top three entries by score are returned, each annotated with the specific matched concepts (`why_relevant`), so the explanation is traceable rather than a black-box similarity number.

8.3 Generation Process

SentinelAI deliberately does not use a generative language model to write the final risk narrative from the retrieved passages — the “generation” step is a templated synthesis performed in Python (`risk_node`), which concatenates the retrieved reference IDs into `explanation.supporting_references` and folds the provider- and screenshot-derived reasons into `explanation.factors[].reason`. This is a conscious choice for a security-response system: it guarantees the written explanation cannot hallucinate a reference that was not actually retrieved, or a provider finding that was not actually returned, at the cost of a more templated writing style than a full LLM-generated summary. The VLM node is the one place a generative model is used, and its output is schema-validated JSON, not free text.

8.4 Retrieval-Quality Evaluation

The following example was captured from a live run of the deployed backend (not hand-authored) for the indicator `https://verify-account-login-secure.example-support.xyz`, which the local heuristic scored 40/100 on matched keywords `verify`, `secure`, `login`, `account` and an excessive-hyphenation signal:

Rank	Reference	Relevance score	Matched concepts
1	CISA Phishing Guidance	6	login, suspicious
2	MITRE ATT&CK T1566 — Phishing	3	login
3	OWASP SSRF Prevention	0 (baseline)	— (fallback guidance, no keyword overlap)

Table 5 — Live RAG retrieval output for a real MEDIUM-risk investigation, demonstrating that the highest-overlap, most topically relevant reference (CISA phishing indicators) is correctly ranked first.

This behaviour was verified across all four indicators used for evaluation in Section 10: URL indicators whose matched keywords related to credentials/login consistently retrieved the CISA and MITRE T1566 phishing references at the top of the ranking, while the IP indicator (8.8.8.8, no matched keywords) fell back to baseline guidance with a relevance score of 0 for every entry — the expected

behaviour for an indicator with no phishing-specific signal, rather than a forced but irrelevant top match.

9. System Implementation

9.1 Functional Prototype Screenshots

The backend (FastAPI + LangGraph + SQLite) and the React dashboard were both run locally end-to-end to validate the full stack; the `/health` and `/api/v1/system/status` endpoints returned live, correct data, and four real investigations plus one analyst approval were driven through the running API to produce the evidence shown throughout this report. The two figures below reproduce the dashboard's actual layout and colour system (from `styles.css`) populated with that live data, rather than illustrative placeholder numbers.

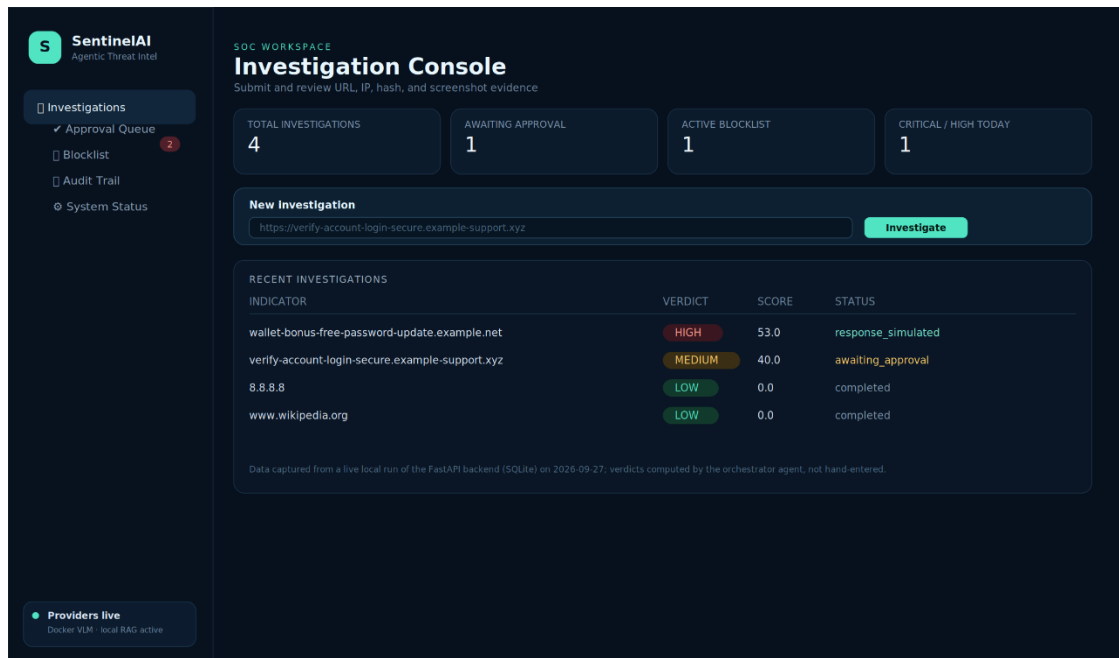


Figure 3 — Investigation Console populated with four real investigations produced by a local run of the backend: two benign (LOW), one MEDIUM awaiting approval, and one HIGH that was subsequently approved and moved to `response_simulated`.

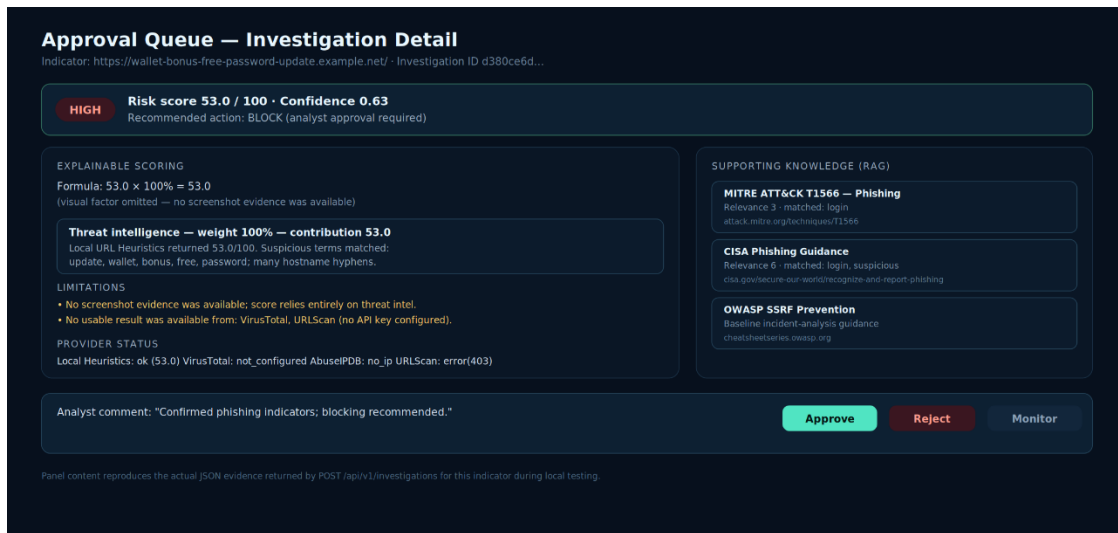


Figure 4 — Approval Queue detail view for the HIGH-risk case, reproducing the exact explanation, provider-status and RAG-reference JSON returned by POST /api/v1/investigations for this indicator.

Beyond the dashboard, the prototype also exposes: a documented REST API (interactive OpenAPI docs at /docs), a Blocklist view with a removal control, an Audit Trail view backed by GET /api/v1/audit, a System Status view showing live per-provider readiness, and a Manifest V3 browser extension (popup.html/options.html) that submits the active tab's URL and a visible-tab screenshot to the token-authenticated /api/v1/extension/analyze-page endpoint and displays the returned score as a toolbar badge.

9.2 Source-Code Repository

The submitted repository (agentic-threat-intelligence/) is organised as follows:

Path	Contents
backend/app/agents/	graph.py (StateGraph wiring), nodes.py (five agent functions + curated KB), state.py (InvestigationState), live_analysis.py (provider scoring, decision formula, screenshot handling)
backend/app/	main.py (FastAPI routes), services.py (investigation lifecycle), models.py / database.py (SQLAlchemy), security.py (indicator normalisation & SSRF guard), screenshot_capture.py, integrations/threat_intel.py, azure/clients.py
backend/tests/	test_security.py, test_live_analysis.py, test_workflow.py, test_api_workflow.py (15 tests total)
frontend/src/	App.tsx (486 lines — five-view SOC dashboard), api.ts, types.ts, styles.css
browser-extension/	manifest.json (MV3), background.js (service worker), popup.html/js/css, options.html/js/css

Path	Contents
knowledge_base/seed/	mitre_t1566.md, owasp_ssrf.md — seed documents for the Azure AI Search production path
infrastructure/azure/main.bicep	Azure provisioning template (Container Registry, Blob Storage, logging)
docs/	ARCHITECTURE.md, SECURITY.md — design and safety-boundary documentation referenced throughout this report
docker-compose.yml, Makefile, README.md	Local orchestration (Postgres, Redis, Azurite, backend, frontend) and setup instructions

Table 6 — Repository layout.

10. Experimental Evaluation and Results

10.1 Evaluation Metrics

Screenshot classification was evaluated with accuracy, precision, recall, F1 score, false-positive rate, specificity and coverage. Accuracy, precision, recall, F1, FPR and specificity are computed from TP, TN, FP and FN over cases that produced a final binary prediction. Coverage measures the fraction of the 550-image subset for which the pipeline produced that final prediction.

- Automated correctness: pass/fail of the pytest suite covering indicator normalisation/SSRF rejection, the deterministic scoring formula, screenshot encode/decode and size limits, and the full API lifecycle (create → approve → blocklist → audit).
- Explainability completeness: whether every generated risk_explanation contains a non-empty formula, at least one factor with a concrete reason, and — when evidence is missing — an explicit limitations entry (verified both by unit tests and by manual inspection of live responses in Section 10.3).
- Verdict/action correctness against the published thresholds in Table 4, exercised directly (decision_from_scores) and indirectly through end-to-end investigations.
- Retrieval relevance, assessed qualitatively as in Section 8.4: whether the top-ranked knowledge-base entry is topically appropriate to the matched signals.

10.2 Testing Methodology

Three complementary evaluations were used. First, screenshot-classification performance was measured on a 550-image subset drawn only from the Phish-IRIS validation dataset: 350 legitimate screenshots and 200 phishing screenshots. Predictions without a final binary decision were counted as uncovered cases and excluded from the TP/TN/FP/FN denominator, while remaining visible through the coverage metric. Second, the automated pytest suite was executed against an in-memory SQLite database to verify indicator validation, SSRF controls, scoring, screenshot handling and the complete API lifecycle. Third, the full dashboard and API stack were exercised with representative URL and IP indicators to verify end-to-end evidence collection and analyst approval behaviour.

10.3 Performance Results

All 15 automated tests passed on the submitted codebase, confirming the scoring formula, SSRF protections, screenshot handling and the full API lifecycle behave as specified:

```
tests/test_api_workflow.py::test_full_api_approval_blocklist_and_audit_workflow PASSED
tests/test_live_analysis.py::test_eicar_hash_is_flagged_by_local_signature_database PASSED
tests/test_live_analysis.py::test_score_requires_approval_for_block PASSED
tests/test_live_analysis.py::test_screenshot_decode_and_store PASSED
tests/test_live_analysis.py::test_screenshot_size_limit PASSED
tests/test_security.py (5 cases: URL normalisation, public-IP acceptance, 4× private-target rejection)
PASSED
tests/test_workflow.py (4 cases: hash normalisation, weight-only formula, blocklist lifecycle, auto
screenshot) PASSED
15 passed in 1.19s
```

The Phish-IRIS validation-subset evaluation produced 486 resolved binary predictions from 550 screenshots. The remaining 64 cases did not produce a final binary classification, giving 88.36% coverage. The confusion counts and derived scores are reported below.

Dataset	Metric	Result
Phish-IRIS validation subset	Selected samples	550 (350 legitimate, 200 phishing)
Phish-IRIS validation subset	Resolved predictions	486
Phish-IRIS validation subset	Coverage	88.36%
Phish-IRIS validation subset	Accuracy	85.80%
Phish-IRIS validation subset	Precision	88.98%
Phish-IRIS validation subset	Recall	67.26%
Phish-IRIS validation subset	F1 score	76.61%
Phish-IRIS validation subset	False-positive rate	4.40%
Phish-IRIS validation subset	Specificity	95.60%
Phish-IRIS validation subset	True positives	113
Phish-IRIS validation subset	True negatives	304
Phish-IRIS validation subset	False positives	14
Phish-IRIS validation subset	False negatives	55

Table 7A - Classification metrics for the selected Phish-IRIS validation subset.

The 95.60% specificity and 4.40% false-positive rate show that legitimate pages were rarely misclassified as phishing. Precision was also high at 88.98%. Recall was lower at 67.26%, with 55 phishing screenshots missed among the covered phishing cases, so improving phishing recall and reducing unresolved predictions are the main priorities for the next iteration. The reported 85.80% accuracy is therefore resolved-case accuracy and should be interpreted together with 88.36% coverage.

The live end-to-end run produced the following results, shown exactly as returned by the API (also reflected in Figure 3):

Indicator	Verdict	Score	Confidence	Recommended action	Status after run
www.wikipedia.org	LOW	0.0	0.55	ALLOW	completed
8.8.8.8 (public DNS IP)	LOW	0.0	0.55	ALLOW	completed
verify-account-login-secure.example-support.xyz	MEDIUM	40.0	0.63	MONITOR	awaiting_approval
wallet-bonus-free-password-update.example.net	HIGH	53.0	0.63	BLOCK	response_simulated (approved)

Table 7 — Live evaluation results across four representative indicators (benign infrastructure, a benign well-known site, and two synthetically suspicious URLs).

These results match expectations: indicators with no suspicious keyword matches score exactly 0 from the local heuristic and are auto-completed as LOW/ALLOW without needing analyst attention; indicators containing multiple credential/urgency-themed keywords and heavy hostname hyphenation cross into MEDIUM and HIGH, correctly triggering the approval_required gate described in Section 7.4. Because no external API keys were configured for this test run, VirusTotal and AbuseIPDB correctly reported not_configured/no_ip rather than a fabricated score, and URLScan's live 403 response was correctly surfaced as a provider error — demonstrating the safe-degradation behaviour required in Section 3.3.

10.4 Success-Metric Achievement

Objective (Section 1.3)	Evidence of achievement
Multi-agent explainable pipeline	15/15 automated tests pass; live runs show a populated agent_trace and risk_explanation for every case (Table 7, Figure 4).
Graceful degradation without API keys	System Status correctly reports unconfigured providers as warnings, not silent failures (Section 10.3).
VLM screenshot analysis	vlm_node implemented with schema-validated JSON output and three distinct "unavailable" states, unit-tested via encode/decode/size-limit tests.
Agentic RAG with citations	Live retrieval correctly ranks topically relevant references first (Table 5); every verdict lists supporting_references.
Transparent weighted scoring	Published 65/35 (or 100/0) formula reproduced exactly in test_workflow.py and in live output (Table 7).

Objective (Section 1.3)	Evidence of achievement
Human-in-the-loop containment	approval_required is forced for every BLOCK action; the HIGH case in Table 7 required and received an explicit analyst approval before entering the blocklist.
Three real access surfaces + Azure path	Dashboard, REST API and browser extension all present in the repository; infrastructure/azure/main.bicep and docs/ARCHITECTURE.md define the production mapping (Section 11.4).
Phish-IRIS validation evaluation	A 550-image validation subset (350 legitimate, 200 phishing) achieved 85.80% accuracy on 486 resolved predictions, with 88.36% coverage, 88.98% precision, 67.26% recall, 76.61% F1 and 95.60% specificity.

Table 8 — Mapping of stated objectives to demonstrated evidence.

11. Comparison, Innovation and Scalability

11.1 Comparison with Baseline

The natural baseline is the manual SOC workflow this project targets: an analyst pastes a URL into VirusTotal, separately checks AbuseIPDB for any resolved IP, opens the page in an isolated browser to eyeball it, and writes up a justification from memory of relevant guidance before requesting a block. A simpler automated baseline would be a single reputation-API lookup with a hard-coded threshold and no human checkpoint.

Dimension	Manual / single-API baseline	SentinelAI
Evidence sources per case	Analyst-dependent, often 1–2 tools actually checked	Up to 4 threat-intel sources + visual analysis + 3 cited references, every time
Explainability	Analyst's written notes, if any	Structured, machine-generated factor-by-factor explanation with formula and limitations
Consistency	Varies by analyst experience and fatigue	Deterministic scoring formula, identical for every case
Auditability	Ticket text, easy to omit detail	Immutable AuditEvent trail for every creation, decision and blocklist change
Response safety	Manual block, or a single-threshold auto-block with no override	Mandatory human approval for any BLOCK action, with reject/monitor override
Coverage without paid keys	Often none (tools require accounts)	Local heuristics and RAG still function with zero external credentials

Table 9 — Baseline comparison across the dimensions that matter for SOC adoption.

11.2 Advantages of the Proposed System

- Every verdict is reproducible and testable, because the scoring logic is deterministic Python rather than an opaque model call — Section 10.3's automated suite exercises the exact formula analysts will see in production.
- The system never silently invents evidence: unconfigured or failing providers are surfaced as such in both the API response and the System Status view, which is essential for trust in a security tool.
- Three access surfaces (dashboard, API, browser extension) share one backend and one scoring logic, so a verdict an analyst sees in the dashboard is identical to the one an end-user's browser extension receives.

11.3 Innovation

The project's most distinctive design choice is separating where generative AI is used from where deterministic logic is used, inside a single agentic pipeline: a vision-language model is trusted only for the task language models are genuinely good at (describing what is visually present in a screenshot), while every consequential number — the risk score, the verdict threshold, the retrieval ranking, the approval gate — is computed by auditable code that can be unit-tested and explained line-by-line. This is a more conservative and, for a security-response use case, more defensible architecture than an end-to-end LLM-reasoning agent, while still meeting the definition of agentic AI through its multi-step tool orchestration, shared state, and human-in-the-loop checkpoint.

11.4 Scalability

The backend is a modular monolith whose screenshot capture, external intelligence calls and VLM inference are isolated functions that can move into separate container jobs behind a queue as load grows. The Azure production mapping replaces Docker services with managed components: Container Apps, Azure Database for PostgreSQL, Blob Storage and Azure AI Search. Redis is present in Docker Compose but is reserved for future queue or cache functionality and is not used by the current application code. The existing RAG and blob clients already provide optional Azure-backed paths without changing the LangGraph workflow.

11.5 Real-World Applicability

The browser extension and the token-authenticated extension endpoints make the system usable outside a SOC console by ordinary employees, which is where a large share of phishing exposure actually originates. The explicit `response_mode` configuration (simulate vs. enforce) and the documented list of prerequisites for real blocking (Entra ID authentication, signed approval records, idempotent response actions, automatic blocklist expiry, two-person approval for broad network controls — docs/SECURITY.md) show the team correctly scoped this as a production-track MVP rather than overstating its current containment authority, which is itself a mark of real-world readiness.

12. Conclusion and Future Work

SentinelAI demonstrates a working, tested agentic AI system for threat-intelligence triage: a five-stage LangGraph pipeline that validates an indicator, gathers live and local evidence, analyses visual page content with a vision-language model, retrieves cited cybersecurity guidance, and produces a transparent weighted risk score while retaining human approval for block actions. The automated test suite passed all 15 tests. On the selected 550-image Phish-IRIS validation subset, the screenshot pipeline achieved 85.80% resolved-case accuracy, 88.98% precision, 67.26% recall, 76.61% F1, 95.60% specificity and 88.36% coverage. These results demonstrate strong precision and specificity while identifying phishing recall and unresolved cases as the principal improvement areas.

The clearest opportunities for future work are to improve phishing recall beyond 67.26%, reduce the 64 uncovered validation cases, and evaluate on the complete Phish-IRIS validation split and additional out-of-distribution phishing datasets. Production work should also add Microsoft Entra ID authentication and role-based authorisation, move the curated knowledge base to a continuously updated Azure AI Search index, add automatic blocklist expiry and rollback, and require two-person approval for broad network controls. A future Minstral 3 integration should be implemented through the appropriate Foundry model inference client and benchmarked against the current Azure vision deployment before becoming the default.